UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Unidad Académica de Algoritmos y Programación
CAO402: Algoritmos II

Profesor: Álvaro Espinoza

Fecha: 04/02/2026

PARCIAL I: PROGRAMACIÓN ORIENTADA A OBJETOS
Período Lectivo 1-2026

Nombre y Apellido: _____ Cedula de Identidad:_____

PARTE I: Lea cuidadosamente la pregunta y seleccione la opción correcta. Se aplicará factor de corrección: 3 respuestas incorrectas eliminarán una correcta, una respuesta en blanco no afectará a las respuestas correctas (1 punto c/u)

a) Dadas las siguientes relaciones: B hereda de A, C compone a A, D tiene una asociación "usa" hacia C. ¿Cuál afirmación es correcta?
   a) `D` puede usar directamente objetos de tipo `A` y `B`.
   b) `C` puede contener objetos de tipo `B`.
   c) `D` y `A` están relacionados indirectamente mediante `C`.
   d) La herencia entre `A` y `B` rompe la composición con `C`.
   e) Ninguna relación aquí es válida en POO.

b) Una clase `X` tiene un atributo `ref` que apunta a un objeto de la misma clase `X`. Además, `Y` hereda de `X`. ¿Cuál afirmación es verdadera?
   a) `ref` pasa automáticamente a ser de tipo `Y` en objetos de `Y`.
   b) `Y` no hereda el atributo `ref`.
   c) `ref` puede apuntar a un objeto de tipo `Y`.
   d) `X` no puede acceder a los atributos del objeto apuntado por `ref`.
   e) Esto genera herencia múltiple implícita.

c) Una clase `A` contiene un atributo `b` de tipo `B`, recibido como parámetro en el constructor. `B` puede existir independientemente de `A`. Esto describe correctamente:
   a) Composición estricta.
   b) Herencia débil.
   c) Agregación.
   d) Asociación de uso.
   e) Polimorfismo estructural.

d) Dada la siguiente jerarquía: `Animal` define métodos públicos y protegidos. `Mamifero` hereda de `Animal`. `Perro` hereda de `Mamifero`. ¿Cuál afirmación es correcta?
   a) `Perro` no puede sobrescribir métodos definidos en `Animal`.
   b) `Mamifero` debe ser abstracta para que `Perro` herede correctamente.
   c) `Perro` hereda indirectamente los miembros públicos y protegidos de `Animal`.
   d) La herencia en múltiples niveles rompe el encapsulamiento.
   e) Es preferible que `Perro` herede directamente de `Animal`.

e) Una clase `A` tiene todos sus atributos protegidos. Una clase `B` hereda de `A`. Una clase `C` compone a `A`. Seleccione la opción correcta:
   a) `C` puede acceder directamente a los atributos de `A`.
   b) `B` y `C` tienen el mismo nivel de acceso a los atributos de `A`.
   c) `B` puede acceder directamente a los atributos de `A`.
   d) Los atributos protegidos solo existen para herencia múltiple.
   e) Cambiar atributos a protegidos viola el encapsulamiento siempre.

f) Una clase abstracta `Figura` define el método `area()`. `Cuadrado` y `Circulo` implementan dicho método. Una variable `Figura f` referencia en tiempo de ejecución a un `Circulo`. Esto es un ejemplo de:
   a) Polimorfismo dinámico.
   b) Sobrecarga de métodos.
   c) Polimorfismo estático.
   d) Encapsulamiento funcional.
   e) Resolución temprana de métodos.

g) Una clase define cinco constructores con distinta lista de parámetros, ninguno heredado. ¿Cuál afirmación es correcta?
   a) Es un ejemplo de polimorfismo estático.
   b) El constructor se selecciona en tiempo de ejecución.
   c) Es un ejemplo de sobreescritura.
   d) Esto solo es posible si la clase es abstracta.
   e) Es herencia múltiple implícita.

h) Una clase `Pedido` necesita trabajar con `Producto` solo dentro de un método, sin almacenarlo como atributo. ¿Cuál relación es más adecuada?
   a) Asociación de uso.
   b) Composición.
   c) Agregación.
   d) Herencia.
   e) Ninguna, debe usarse siempre agregación.

i) Dada una clase `A` que hereda de `B`, y una clase `C` que hereda de `A`. ¿Cuál afirmación es correcta?
   a) `C` puede acceder directamente a los atributos protegidos de `B`.
   b) `C` hereda todos los atributos privados de `B`.
   c) `B` puede acceder a los métodos de `C`.
   d) `A` no puede usar métodos definidos en `B`.
   e) La herencia solo es transitiva para atributos públicos.

j) ¿Cuál de las siguientes situaciones rompe conceptualmente el encapsulamiento?
   a) Permitir acceso directo a atributos internos desde clases hijas sin control.
   b) Usar getters y setters para acceder a atributos privados.
   c) Ocultar la implementación interna de un método.
   d) Usar métodos privados para lógica interna.
   e) Definir interfaces públicas claras.

PARTE II: Modelado de Clases: Realice el diagrama de clases correspondiente a la siguiente situación **(5 pts)**:

Los Asquerositos de Alvaro

Un emprendimiento gastronómico venezolano conocido como **"Los Asquerositos de Álvaro"**, el cual opera como un **puesto callejero de perros calientes** (no un local formal, pero con abundancia de salsa), desea desarrollar un sistema orientado a objetos que le permita llevar el control de su negocio. Se conoce que el puesto maneja información de los clientes que realizan pedidos, de los perros calientes que se ofrecen, de los ingredientes disponibles, de los combos que se comercializan y de los pedidos realizados. De cada cliente se desea conocer su nombre completo, cédula de identidad, fecha de registro y la cantidad total de pedidos realizados. Se sabe que un cliente puede realizar uno o más pedidos a lo largo del tiempo, y que cada pedido pertenece a un único cliente.

El puesto ofrece distintos tipos de perros calientes. De todo perro caliente se conoce su nombre, precio base y tamaño. Se sabe que existen perros calientes clásicos, especiales y monstruosos, y que cada tipo puede poseer información adicional distinta; en particular, los monstruosos poseen una cantidad mínima obligatoria de ingredientes. Asimismo, el sistema debe manejar la información de los ingredientes. De cada ingrediente se conoce su nombre, tipo (salsa, proteína o acompañante) y costo adicional. Un perro caliente puede contener uno o más ingredientes, y un mismo ingrediente puede formar parte de múltiples perros calientes.

Adicionalmente, el puesto ofrece combos conformados por un perro caliente, una bebida y un acompañante. De cada combo se conoce su nombre y precio total. Un combo no puede existir sin un perro caliente asociado, aunque un perro caliente puede venderse de manera independiente. En cuanto a los pedidos, de cada uno se desea registrar un código único, la fecha y hora de realización y el cliente que lo efectuó. Un pedido puede incluir uno o más perros calientes y cero o más combos. El sistema debe ser capaz de calcular el monto total del pedido considerando precios base, ingredientes adicionales y combos. Finalmente, se desea conocer en todo momento cuántos clientes están registrados, cuántos pedidos se han realizado en total y qué ingredientes han sido utilizados en un pedido específico. El sistema debe poder ser gestionado desde la clase Pedido, sin utilizar estructuras externas auxiliares.

PARTE III: Implementación (5 pts):

Caja Registradora de "Los Asquerositos de Álvaro"

Se desea implementar un sistema básico de cobro para el puesto de perros calientes **"Los Asquerositos de Álvaro"**. Para ello, se sabe que el puesto vende **ítems de venta** de dos tipos: **Productos** y **Combos**. Se requiere diseñar clases que permitan calcular el monto total de un pedido de forma orientada a objetos usando **herencia**.

Se sabe que todo ítem de venta posee un **nombre** y un **precio final**, pero el precio final se calcula de forma distinta según el tipo de ítem. Por esta razón, se requiere una clase abstracta `ItemVenta` con al menos: el atributo nombre, y el método abstracto `calcularPrecioFinal(): real`.

Un `Producto` es un ítem de venta que tiene: nombre, precio base y una cantidad de extras (entero). Cada extra incrementa el precio en un monto fijo `costoExtra` (real) que es igual para todos los productos (asuma que este valor se le pasa al constructor del producto). Por lo tanto, el precio final de un producto es:

`precioBase + (extras * costoExtra)`.

Un `Combo` es un ítem de venta que está conformado por: un `Producto`, una bebida (cadena) y un descuento fijo `descuento` (real). El precio final del combo se calcula como:

`precioFinalDelProducto - descuento`, y si el descuento hace que el precio final sea negativo, entonces el precio final del combo debe ser 0.

Por otra parte, un `Pedido` contiene un arreglo de `ItemVenta` (es decir, puede contener tanto Productos como Combos), y un entero con la cantidad de ítems presentes. Se requiere que `Pedido` implemente:

1. Un constructor que reciba un arreglo de `ItemVenta` y lo almacene internamente.
2. Un método `totalPagar(): real` que retorne la suma del precio final de todos los ítems del pedido.
3. Un método `itemMasCaro(): ItemVenta` que retorne el ítem con mayor precio final (si hay empate, retorne cualquiera).

Por último, es obligatorio declarar getters y setters para todos los atributos principales. No es necesario implementar la lógica interna de los setters/getters, pero deben existir.

Nota: Se asume que existe la función `compararCadena(cadena1, cadena2): logico` para comparar nombres si lo considera necesario.

……

Sobró espacio, te regalo un código para que te recuerdes de que el profe te quiere :)

```
#include <iostream>
using namespace std;

int main() {
    cout << "Si tu solucion tiene 15 ifs para saber si es Producto o Combo...\n"
         << "felicidades: inventaste programacion estructurada con cosplay de
POO.\n";
    return 0;
}
```

“La simplicidad es un prerrequisito para la fiabilidad.”

— *Niklaus Wirth*